Hardening the Exact-Cell KV Window for `--kv q4lm8v`: Design, Decisions, and a Measurement-First Validation

Matthias Kronenberger

October 2026

Abstract

The exact-cell KV window is an opt-in memory/accuracy point for Strata's 4-bit Lloyd KV cache (`--kv q4lm8v`). A small resident ring of recent cells and a set of sink cells are kept in fp16 while the rest of the cache remains 4-bit. With S=256 recent cells, R=64 sinks and the automatic prompt chunk capped at B=6,144, the production point uses 5.29 bit/cell (1,016 MiB at 128K) against `int8`'s 8.25 bit/cell (1,584 MiB), at equal prefill throughput and decode inside run-to-run spread. This paper describes the design, the correctness hardening (A1–A5), and a measurement-first validation. Quality is scored teacher-forced against an fp16 KV reference on three fixed corpora (2K, 4K, and 128K — the 128K point on two texts), reporting mean KL, p99 KL, top-1 agreement and perplexity with paired per-segment statistics; the KL is computed over the reference's top-64 tokens, renormalized (Section 6.3). The window removes most of the plain 4-bit base's error on the cwe task: 73% of its gap to `int8` at 4K, and across the two 128K texts 34–58% of the plain base's tail mean KL. At 2K its mean KL is indistinguishable from `int8`'s (0.00702 vs 0.00503, $t = -1.92$, window ahead in 2 of 8 segments); at 4K `int8` is significantly better on the mean ($t = -3.55$, 7 of 8 segments) but indistinguishable in the answer region; at 128K `int8` keeps the better p99, the last 1,024 positions and the whole segment on both texts. The result is a 36% KV-memory saving with a small, measurable KL cost, not an accuracy win over `int8`. The `int8` default path stays byte-identical (dump-logits SHA against the previous release); AMD HIP is measured, and CUDA/SYCL builds plus a full cell-to-row parity test remain review gates.

1 Introduction

At long context the KV cache is a large, fixed cost: at 128K tokens the `int8` default reserves 1,584 MiB before any expert weights are loaded. A 4-bit KV cache reduces that, but quantizing every cell uniformly costs accuracy on long documents: earlier 4-bit KV work measured document perplexity about 8% and 12% above the *fp16 reference* at 1K and 8K context (about 7% and 13% above the `int8` default; Q2_0, `q4_0`) [16, 14]. The exact-cell KV window is an attempt to keep the memory saving while protecting the cells that matter most.

The memory landscape assumed here is not the datacenter one. On Hopper- and Blackwell-class NVIDIA hardware, an FP8 KV cache is the production two-times point: vLLM reports degradation within 1–2 accuracy points on its reasoning-suite benchmark and aggregate recovery on prompts up to a million tokens — a suite-specific bound whose longest buckets show visible variance, and whose head dimension of 256 is where FP8 prefill still regresses [13]. A consumer AMD GPU driven through HIP has no FP8 attention path; on such hardware the block-quantized `int8` default is the deployed two-times saving, and 4-bit is the frontier. That is the constraint the window addresses.

The idea is simple. The most recent S cells and the first R cells (attention sinks) of each cache-carrying attention layer — on the measured model, the 12 full-attention QSA layers of 48 (Section 3) — are kept in fp16 in a resident side pool; older cells stay in the 4-bit Lloyd format. The window thus spends exact bytes where the model is most sensitive and leaves the long tail compressed. It is opt-in (`--kv q4lm8v` plus `STRATA_KV_WINDOW=1`) and never changes the default `int8` path.

The implementation was functional in principle, but several integration points could fail silently: the host-side prompt chunking, the interaction with KV streaming, the stability of the window grid across model states, and the meaning of uninitialized ring entries. We treated these as correctness issues, not performance issues, and fixed them behind the same opt-in flag.

This paper positions the window against prior and current KV-cache compression work (Section 2), describes the design and its production point (Section 3), records the alternatives weighed against it and why each was left out (Section 4), documents the hardening decisions and the evidence that the default path is untouched (Section 5), sets out the measurement methodology (Section 6), and reports results at 2K, 4K and 128K with a discussion of where the point fits (Sections 7–8). The emphasis is on the methodology and on what the measurements do and do not show; file-level implementation detail is deliberately kept out.

2 Related Work

KV-cache compression splits into *eviction* and *quantization*. Eviction keeps a subset of cells and drops the rest: H2O [7] accumulates attention as heavy hitters, SnapKV [5] selects per-head clusters from an observation window, and PyramidKV [6] allocates larger budgets to lower layers. Quantization keeps every cell at a lower bit width: KIVI [2] shows tuning-free 2-bit quantization with per-channel K and per-token V plus an fp16 residual over recent tokens; KVQuant [3] adds per-channel keys, pre-RoPE key quantization and non-uniform datatypes to reach sub-0.1 PPL degradation at 3 bits; GEAR [4] corrects residual error with a

System	Point	Documented outcome
KIVI [2]	2b	per-channel K, per-token V
KVQuant [3]	3b per-channel + pre-RoPE	<0.1 PPL degradation (Wikitext-2/C4)
GEAR [4]	4b + correction	low-rank + sparse
RotateKV [9]	2b	rotation-based
KVTuner [10]	3.25–4.0b	layer-wise pairs, nearly lossless
SnapKV [5]	eviction	per-head clusters
H2O [7]	eviction	attention-accumulated heavy hitters
PyramidKV [6]	eviction	12% of cache matches full cache (LongBench)
`llama.cpp` [12]	q8_0 / q4_0	block-quantized, per-cache type
`vLLM` [13]	FP8	≤1–2 acc. points up to 1M-token prompts

Table 1: Documented operating points from the literature and production runtimes. All entries are the cited works' own published results on their own hardware and models; they are not comparable across rows.

low-rank plus sparse-outlier decomposition. Rotations make low-bit quantization robust to channel outliers: QuaRot [8] and RotateKV [9] apply orthogonal or Hadamard rotations before quantization. Strata's 4-bit base quantizes per cell after exactly such an orthonormal Walsh–Hadamard rotation, so it belongs to that family. Its production split stores K with 16 Lloyd levels and V with 8 inside the same 4-bit block; the paired K-vs-V contrast on Swift did not support a K-is-more-sensitive story (t=+0.05, McNemar p=1.0 [17]), so the split is a byte/codebook choice, and KVTuner [10]'s layer-wise pairs are a different, so-far unmeasured axis. For token-precision co-tuning, [11] studies the token/bit trade-off. Table 1 collects the documented operating points in one view.

In production, `llama.cpp` ships block-quantized KV (q8_0, q4_0) selected per cache [12], and `vLLM` ships an FP8 KV-cache path whose reported degradation stays within 1–2 accuracy points on its reasoning-suite benchmark (the caveats already stated above apply) [13]. Strata targets a corner where neither path exists—one consumer GPU through HIP, no FP8 attention kernels—so its block-quantized `int8` default already provides the two-times point, and this paper moves toward 4-bit.

The window itself is a quantization-side design: it keeps the whole context addressable (no eviction, unlike [7, 5, 6]), and it spends fp16 bytes on a small recent-plus-sink overlay. The recent-token residual is KIVI's [2] and the sink retention is StreamingLLM's [1]; what this paper adds is the overlay on top of a block-quantized Lloyd base on this engine, the hardening around it, and the paired-statistics discipline used to read it. The fair competitor we do not measure here is a per-channel-K arm in the KIVI/KVQuant style on the same engine; it is listed as follow-up work.

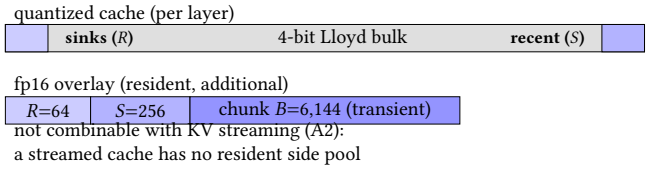


Figure 1: Cache anatomy per layer. The blue slivers mark *which* token positions the resident fp16 overlay additionally covers — sinks at the cache's start, recent cells at its end; sliver widths are exaggerated, the position order is exact, and the sliver colors match the overlay segments in the second bar. The cells themselves remain quantized in the 4-bit base; the copied fp16 rows form an *additional* side pool, not a hole in the cache.

Code and documentation for the compared systems are listed with the references [1, 2, 3, 10, 12, 13]. Strata's own measurement provenance lives in the repository and is described in Section 6.6.

3 Design of the Exact-Cell Window

We name the machine (a static reading of the compiled `geometry.hpp`, not a measurement): the measured model is Qwen3.8-Flash-Next, a hybrid of 48 layers in which every fourth layer (layers 3, 7, …, 47) is a full-attention QSA layer with the KV cache this paper quantizes — 12 such layers; the other 36 carry no cache of this kind. A QSA query does not read the whole cache: its indexer selects at most 2,048 cells (plus block rounding) per query. The window changes the *precision* of selected cells that fall in the sink or recent set, not the selection itself — we found no code that forces sinks or recent cells into the selection, and verifying that is on the follow-up list (Section 10). "Keeps the cells that matter most" (Section 1) is therefore a precision statement about selected cells, and the eviction comparison of Section 2 is about storage, not about which cells are attended.

Three sizes need distinct words. A *token position* is one context token in one layer; in the 4-bit base its stored cost is 576 B per layer (K and V together). A *scalar element* is one K or V number — 1,024 per token position (2 KV heads × 256, K and V) — and the rates of this paper (4.50, 5.29, 8.25 *bit/cell*) are bits per element. In the prose, *cell* means one token position's K/V entry in one cache-carrying layer, and a *row* is that position's storage within the layer's cache. `int8` names the default's block format: 8-bit content with one fp16 scale per 64 elements, 8.25 effective bit/element.

3.1 Operating point

The production point is S=256, R=64 and the automatic prompt chunk capped at B=6,144. Table 2 compares it with the default. The window costs 0.79 bit/cell more than the plain 4-bit base (4.50 bit, 864 MiB) and frees 36% of the KV memory against `int8`.

3.2 Window geometry and cost

The 4-bit base stores K at 16 levels and V at 8 levels in 18-byte blocks, 576 B per cell and layer. The window keeps a side copy of the sink and recent cells in fp16. The side copy is

KV mode	Bits/cell	KV at 128K
`int8` (default)	8.25	1,584 MiB
plain `q4lm8v`	4.50	864 MiB
`q4lm8v` + window 256/64	5.29	1,016 MiB

Table 2: KV footprint at 128K (131,072 token positions, the 12 full-attention QSA layers of the hybrid model; Section 3). `int8` stores 8-bit content plus block scales; its 8.25 is the effective rate. The window figure includes the resident ring at B=6,144; the plain figure is from the byte model.

additional to the quantized row, not a replacement: the exact cell is 16 bits, not 16 − base bits. The resident ring holds R sink rows, S recent rows and B rows for the current prompt chunk, so the marginal cost of a windowed run is

$$\text{bits/cell} \;\approx\; 4.50 \;+\; \frac{16\,(R+S+B)}{L}, \tag{1}$$

where L is the context length in cells. All three terms are resident memory: the ring is sized for the prompt chunk, so the B term dominates the per-cell average at this L (0.75 of the 0.79 added bit/cell at 128K). At 128K with R=64, S=256 and B=6,144 this gives 5.29 bit/cell and 1,016 MiB.

The chunk B follows `--prefill`. A larger automatic chunk would need a larger ring, and the host's ring batch cap (`kKvWinBatchMax`) is 6,144 cells, so a windowed run caps the automatic chunk there and says so. A ring the user sizes explicitly is left alone; a chunk that does not fit is refused loudly. The break-even against `int8` is about 28K cells: solving $16\,(R{+}S{+}B)/L = 8.25 - 4.50$ gives $L \approx 27{,}600$ at B=6,144, and at 32K the window is still cheaper than `int8` (7.67 against 8.25 bit/element [17]). Setup offers the window only from 32K upwards — above the break-even, where the memory direction is the right one.

The exact cells must stay resident. A streamed KV cache has no always-resident side pool, so the window and KV streaming do not combine (A2). The window is also orthogonal to the level table: it can be enabled on other 4-bit modes, but the production point is `q4lm8v` with 256/64.

4 Design Alternatives Considered

None of the alternatives in this section was measured on this engine. The section records why each was left out; the reasoning is static analysis, not measurement, and every item is a candidate follow-up arm.

Per-channel key quantization. KIVI and KVQuant quantize K along the channel axis because the key error concentrates in outlier channels [2, 3]. Strata's base quantizes per cell after an orthonormal Hadamard rotation, which spreads that outlier energy across channels; whether the rotation makes the cheaper per-cell axis competitive with the per-channel axis on this model is exactly the unmeasured question. It would change the block writer and reader, not the window.

Pre-RoPE key quantization. KVQuant quantizes K before the rotary positional embedding, because the rotation multiplies the quantization error [3]. Strata's Hadamard rotation is also pre-quantization; whether the engine's K path can quantize before RoPE instead of after is a kernel-layout question that has not been examined.

Non-uniform levels. KVQuant derives sensitivity-weighted datatypes [3]; our K uses 16 uniform Lloyd levels and V uses 8. Non-uniform levels are orthogonal to the window and would change the level tables only.

Low-rank plus sparse correction. GEAR keeps a correction term for the residual error [4]. The window's fp16 overlay is a dense-but-local correction: full 16-bit precision on the ring's cells. A distributed low-rank correction would cover the whole cache at a smaller byte rate, at an added latency cost; both would need new kernel shapes.

Layer-wise precision pairs. KVTuner reaches a nearly lossless 3.25-bit point with different precision per layer [10]. Strata's KV format is global per cache; per-layer level tables are representable in the block layout but add configuration space that setup would have to explain to users.

Eviction on top of the 4-bit base. SnapKV/H2O-style eviction is a larger memory win with a different cost: cells leave the cache [5, 7]. The window keeps the whole context addressable — the A-series treats a silently unavailable exact cell as a fault — so eviction is out of scope for this point; a combined mode would be a separate opt-in with its own hardening.

A streaming-compatible sink pool. A2 refuses the window under streaming. A tiny sink-only side pool (R cells, no recent ring) could in principle combine with a streamed cache, because sinks never scroll far from position 0. We did not build it: the sink budget interacts with the streaming host copy, and the contract prefers loud refusal over a new silent-failure surface. Untested.

5 Hardening: Preventing Silent Failures

The guiding principle is that a silent misread is worse than a loud refusal. When the window cannot guarantee an exact read, it should stop, refuse the configuration, or fail an invariant—not fall back quietly to a quantized value that the caller believes is exact. Table 3 summarizes decisions A1–A5, including A4 as the visible open item.

Two decisions deserve emphasis. A2 is enforced twice—once in setup, where most users will meet it, and once in the engine, where a hand-edited configuration can still arrive. A3 is deliberately a freeze-and-assert rather than a full state-layout refactor; the assert is active in debug builds, and a

ID	Observed problem	Decision	Evidence
A1	The automatic prompt chunk could exceed the ring's batch cap, so an append in the middle of a long prompt aborted the run.	Cap the automatic chunk at the ring's batch maximum (6,144) when the window is active and the user did not set a size; cover the generate and serve paths.	The pre-fix failure is the logged one (an 8,191-token prompt, `--prefill auto:16384`, aborted); post-fix completion is the capped path's behavior (static reading of `generate.cpp`). An explicitly undersized ring is still refused.
A2	Setup could write a streamed KV cache beside the window. A streamed cache has no always-resident side pool, so a promised exact read could silently become quantized.	Setup never writes the pair and explains why; the engine refuses the pair before loading anything.	The pair is refused before load with exit code 2; window alone and `int8` with streaming both run [14].
A3	The (S, R, B) grid could drift between the main state and a later-carved MTP/twin state. Different grids on the same contract are incorrect.	Freeze the grid after the prompt chunk and assert stability on later batch changes.	The grid stays stable; the default logits SHA is unchanged. The check is a debug assert (release builds warn and continue); probe results are not in the measurement history.
A4	The window's write/read paths have no full parity test across all shapes, so a shape-specific misread would be silent.	**Open item.** Dump every window row at write time and re-read it through the window reader on every shape: ring wrap, stale tags, sinks, batch boundaries, empty tags.	Not yet run; the test is specified in the repository and is a review gate.
A5	Zero-initialized ring rows were tagged as cell 0 and could be read as a valid exact cell.	Initialize empty tags to a sentinel; a stale or mismatched tag falls back to the quantized row.	Empty-state semantics are correct and the default path is unchanged.

Table 3: Hardening decisions A1–A5. The A4 row is the open follow-up item (a full cell-to-row parity test across all shapes), a review gate rather than a shipped fix.

release build prints a warning and continues. A4 is the one item in the A-series that is still open.

6 Evaluation Methodology

6.1 Corpora

Three fixed teacher-forced corpora are used. The 2K set is the RULER *complex word extraction* task in eight contiguous segments, 16,351 scored positions. The 4K set uses the same task with eight segments of 4,096 tokens, 32,746 positions. The 128K set is two texts from the same generator (same seed, alpha and 2,621 code words, drawn separately) [14]; the first is one 131,063-token segment, 131,062 scored positions, scored on the last 4,096 positions and on the whole segment, and the second is scored on the same regions (Section 7.3). The three sets are different texts; numbers are not comparable across lengths, and the 4K set is not a length-scaling experiment for the 128K set.

6.2 Reference and scoring

The reference is fp16 K/V with fixed Swift 1.5 IQ3_XXS weights. The model is given the reference text and the log-probabilities are read; there is no free generation. Each arm is a separate engine run, and the harness records per-position log-probabilities and compares them offline, so KL, top-1 and PPL come from the same pass. The KV cache of the prefill is quantized along with the decode cache, so the absolute gaps are internal Strata numbers and are not directly comparable with field numbers measured decode-only. Each quality run uses one expert slot and `--prefill 512` on all three corpora, and the text is scored through `--short-read` (one decode step at a time, so a scored token's exact set is the $R{+}S$ overlay, not the production chunk). The batched B=6,144 prefill was not quality-scored; that it scores the same is a static argument, not an A/B [17]. The runtime runs use the automatic expert cache and a mixed prompt set.

6.3 Metrics

Four metrics are reported. *Mean KL* is the Kullback–Leibler divergence $D_{\mathrm{KL}}(p_{\mathrm{ref}} \,\|\, p_{\mathrm{arm}})$, from the fp16 reference distribution to the arm's distribution, in nats per position; zero means identical, and lower is better. As the harness computes it for these tables, the KL is truncated to the reference's 64 most likely tokens: both distributions are renormalized onto that set, and a token the arm's own 64-token list omits contributes the arm's 64th log-probability as a floor [15]. This is not the KL of the two full next-token distributions — mass outside the reference's top-64 enters only through the floor — and the numbers here are comparable only within this definition. The direction penalizes an arm that puts little probability mass where the reference has mass, which is the kind of error a downstream sampler inherits; mass the arm places on its own private modes is invisible to it. The reverse direction, a full-vocabulary rescore (which would be a new scoring pass), and the truncation boundary itself were not measured; the ranking could differ under any of them. *p99 KL* is the 99th percentile of the per-position KL under the same truncated definition: it exposes a heavy error tail that a good mean can hide, and the isolated 17-nat positions of

Table 8 carry a floor's signature — the arm's top-64 list missing a reference mode — so they should not be read directly as full-distribution events. *Top-1* is the fraction of positions where the arm's argmax equals the fp16 argmax; it is a coarse agreement measure, not target accuracy. *Perplexity* is exp of the mean negative log-likelihood of the true tokens. Perplexity is reported for context only: quantized-KV error tends to damp the distribution, so a quantized arm can score *below* the fp16 reference while its $D_{\mathrm{KL}}(p_{\mathrm{ref}} \,\|\, p_{\mathrm{arm}})$ is positive. KL, not perplexity, is the decision metric. The p99 entries are point estimates; at these position counts their sampling error is much smaller than the spread between arms, but it is not quantified here — a bootstrap over positions (computed on this truncated KL) is the cheapest missing piece and needs no new runs.

6.4 Paired statistics

Both arms see the same positions. We compute the KL difference per segment, then average over the eight segments. The text cancels, so the spread is much smaller than in a comparison of two independent means. We report the paired t statistic and the number of segments whose difference carries the stated direction, which each result names explicitly. With eight segments (seven degrees of freedom), $|t| \gtrsim 2.4$ is roughly $p < 0.05$ two-sided and $|t| \gtrsim 3.5$ roughly $p < 0.01$. A large t with an unfavorable sign count is not a result; the sign count is read alongside every headline number. The 128K set is one segment, so its eight 4K blocks are adjacent and not independent; its t is a rough reading, not a segment-pair test.

6.5 Runtime, memory and correctness gates

Runtime is a same-day A/B with a fresh server per arm and two alternating rounds; the engine's own prefill and decode counters are read, and table entries are medians of the two rounds. The reported runs use a mixed prompt set — short and long prompts over the same serving session — whose per-prompt results are documented in the measurement report [17]; the hit rate column is the expert-cache hit rate over that set (the fraction of expert lookups served from the resident cache). Reported memory is KV bytes at 128K and resident expert slots. The correctness gate is byte-identity: the `int8` default's dump-logits SHA must be identical across runs, and every window change must leave it unchanged. Targeted probes reproduce the former crash and the refusal cases; kernel parity tests compare the window writer and reader bitwise; needle tests at 32K and 128K check that retrieval does not regress.

6.6 Provenance and reproducibility

Every number in this paper is traceable to the repository's measurement history: the window work — the 2K/4K quality tables, the runtime A/B and the first 128K text — is documented in `docs/KV_QUANT_WORKPACKAGE.md` [17], the authoritative byte model and quality section in `docs/DETAILS.md` [14], and the plain-4-bit document-perplexity measurement in `bench/results/2026-09-27-kv-q4` [16]. The cwe corpora are RULER complex-word-extraction texts generated by the repository's quant-quality harness (fixed segment structure and scored positions per section above); the harness records the per-position log-probabilities the statistics are computed from. No new runs were produced for the quality tables; where a figure is byte-model rather than runtime-observed, the table captions say so. The one run added for this revision is the 2026-10-24 needle pair (Section 7.5); its JSON and engine logs (`.txt`; `*.log` is gitignored) are in the tree [18]. Two provenance limits are named rather than smoothed over. First, the workpackage runs behind Tables 4–8 used engine build 0.1.44.1, while `DETAILS.md` was written under a 0.1.47 Windows–HIP build; it carries the first text's tail numbers to the printed digit plus the second 128K text, whose raw per-position log is not in the tree (Section 7.3). Second, `docs/QUANTS.md` documents the top-64 KL convention for the weight-quantization yardstick, not the cwe protocol, and the window's raw `bench/results` directories hold no logs in this tree — the workpackage on that branch is the report.

7 Results

7.1 Memory and speed

Table 4 gives the 128K runtime point. The production arm (256/64, B=6,144) has the same prefill throughput as `int8`, decode 1.0 token/s lower with changing signs across rounds (inside the run-to-run spread), and frees 568 MiB of VRAM at 128K, which the expert cache converts into 416 more resident expert slots and a slightly higher hit rate. These are capacity figures, not speed wins: serving throughput at concurrency was not measured. The B=512 arm is cheaper (4.60 bit, 884 MiB) but prompts at 105 token/s, about five times slower, so it is an offline/batch point. The 512/128 arm is not chosen: at 4K its quality is statistically identical to 256/64 (paired t=+0.02, Table 6), it was not quality-scored at 128K, it costs 7 MiB more (Table 4), decodes about 0.85 token/s slower, and its apparent prefill advantage did not reproduce on the needle prompts.

7.2 Quality at 2K and 4K

Tables 5 and 6 report the short-context quality; all quality statements in this section are for the cwe task and this model, and are not length-scaling claims across the different corpora. At 2K the window's mean KL is 0.00702 against `int8`'s 0.00503, but the paired difference is not significant ($t = -1.92$ in `int8`'s favor; the window is ahead in only 2 of the 8 segments). The window is also close in the tail: p99 0.0835 against 0.0733. It removes most of the plain 4-bit arm's error (0.01791) and lands near `int8`.

At 4K `int8` is significantly better on the mean (0.01688 against 0.02090; paired $t = -3.55$, and 7 of the 8 segments favor `int8`), and the window closes 73% of the plain-arm-to-`int8` gap (0.01093/0.01494). In the small answer region the two are not distinguishable ($t = -0.51$; descriptive, small region). The 512/128 form is statistically identical to 256/64 ($t = 0.02$), so the smaller form is kept.

Arm	Bit	MiB	Slots	Prefill	Decode	Hit
int8	8.25	1,584	3,033	526.1	**35.2**	0.74
256/64, B=6,144	5.29	1,016	3,449	528.0	34.2	0.76
256/64, B=512	**4.60**	**884**	3,496	105.0	34.5	0.76
512/128, B=6,144	5.33	1,023	3,412	**634.0**	33.35	0.77

Table 4: 128K runtime (Swift 1.5 IQ3_XXS, RX 9060 XT 16 GB, Windows HIP; engine build 0.1.44.1, `--spec 4` MTP drafting on). Prefill and decode in tokens/s; decode is the serving rate under drafts, so the spread includes draft acceptance, not only the KV format. Medians of two same-day alternating rounds over a mixed prompt set. The 512/128 prefill advantage did not reproduce on the needle prompts and is shown for completeness.

Arm	Top-1	Mean KL	p99 KL	PPL
fp16 (reference)	—	0	—	2.999
int8	99.35%	**0.00503**	**0.0733**	3.004
plain q4lm8v	98.92%	0.01791	0.2793	3.024
+ window 256/64	99.33%	0.00702	0.0835	3.003
+ window 512/128	**99.39%**	0.00623	0.0743	**2.995**

Table 5: Quality on cwe-2k (16,351 positions, fp16 K/V reference). Top-1 is agreement with the fp16 argmax; KL in nats/position.

Arm	Top-1	Mean KL	p99 KL	PPL
int8	**99.05%**	**0.01688**	**0.1888**	2.860
plain q4lm8v	98.40%	0.03182	0.5785	2.888
+ window 256/64	98.84%	0.02090	0.2811	2.858
+ window 512/128	98.95%	0.02088	0.2822	2.858

Table 6: Quality on cwe-4k (32,746 positions). cwe-4k is a different text from cwe-2k; the two tables are not comparable across sets.

7.3 Quality at 128K

Table 7 reports the tail of the *first* 128K text, and Table 8 the other regions. On the last 4,096 positions the window's mean KL is 0.00663 against int8's 0.00959, a 31% lower mean, but the paired eight-block t is −0.74: the lead is not significant and comes from two blocks in which int8 has its own spikes. The robust reading favors int8: it has the better p99 (0.01655 against 0.02977), the better last 1,024 positions (0.00265 against 0.00444), and the better whole segment (0.10715 against 0.13312); the region table also shows a lower answer-region KL for int8, but that region has 27 positions, so the number is descriptive only and we do not use it as an argument. The window still beats the plain 4-bit base clearly: mean KL 0.00663 against 0.01587 (58% lower), whole segment 0.13312 against 0.15758, and p99 0.0298 against 0.0308. The 128K result is therefore the same story as 4K: the window recovers much of the base's loss and approaches int8, but does not overtake it.

The second text. The 128K generator produced two texts (same seed, alpha and 2,621 code words, drawn separately); the second — printed in DETAILS.md but not in the workpackage's quality block — changes the tail story [14]. On its last 4,096 positions int8 wins the mean (0.00324 against the window's 0.00437; plain 0.00664) and the p99 (0.0165 against 0.0210); here the window closes 34% of the plain arm's tail KL rather than 58%, and int8 also wins the whole segment (0.06435 against 0.07524 and plain 0.08834). Stable across both texts: the window clearly beats the plain 4-bit base and int8 wins the whole segment; the gap between the two quantized arms is not consistent even in sign, which is why the robust reading rests on the p99, the last 1,024 positions and the whole segment rather than on the tail mean. Provenance caveats: DETAILS reports the first text's tail under its 0.1.47 build label with the digits of the workpackage's 0.1.44.1 table (rerun or retold is not recorded), and the second text's raw per-position log is not in the tree, so its numbers stand as a reported reading, not a re-runnable table.

7.4 Reading the results

Three conclusions follow, all within this task family and this model. First, the window does what it was designed to do: it removes most of the plain 4-bit base's error at every length tested, and at 2K it is statistically indistinguishable from int8 on cwe-2k. Second, it does not replace int8 as the quality default. At 4K int8 is significantly better on the mean; at 128K it keeps the p99, the last 1,024 positions and the whole segment on both texts, while the tail mean favors each quantized arm on one text (inside the text-to-text spread). The window saves 36% of the KV memory at the price of a small, measurable quality gap that the mean KL alone can hide. Third, perplexity is not a useful discriminator here: the 128K PPL values (2.4052, 2.4079, 2.4029) all sit below the fp16 reference (2.4149) while every KL is positive. Reporting PPL without KL would reverse the ranking.

7.5 Correctness gates

The default path is unchanged: the int8 dump-logits SHA (A014F504. . . BA052) is byte-identical to the pre-window release for `--kv int8` and `--kv fp16` [14]. The former crash config now completes under the 6,144 cap (the failure is the logged pre-fix one; the completion is the capped path's behavior, Table 3); an explicitly undersized ring is still refused; the window-plus-streaming pair is refused before loading anything with exit code 2. Kernel parity tests pass 12/12, and the workpackage needle gate is 18/18 at 32K and 128K (6/6 on plain, 256/64 and 512/128; saturated, no regression and no separation) [17]. On 2026-10-24 the production pair was asked again, engine 0.1.47, same card, Swift 1.5 IQ3_XXS, `--spec 4`, chunk 6,144: int8 and the 256/64 window, the same six needles (32K and 128K, depths 10/50/90%, one code word, greedy). Both found 6/6, and each answer is the planted word [18]. Still saturated: no regression, no

Arm	Bit @128K	KV MiB	Mean KL	p99 KL	Top-1	PPL
int8	8.25	1,584	0.00959	**0.01655**	**99.73%**	2.4052
256/64	**5.29**	**1,016**	**0.00663**	0.02977	**99.73%**	2.4079
plain q4lm8v	4.50	864	0.01587	0.03079	99.51%	2.4029

Table 7: Quality on the *first* cwe-128k text (Section 7.3), last 4,096 positions (context 126,967–131,062). The fp16 reference has KL 0 and PPL 2.4149; target-token top-1 is 79.74%. This p99 covers the tail region only and is not comparable with the whole-corpus p99 of Tables 5 and 6; over the whole 128K segment the p99 is 3.66–5.55 at all three arms (Table 8), driven by isolated hard positions. The 864 MiB of the plain arm is from the byte model, not a runtime reading.

Region / metric	int8	256/64	plain
Last 1,024, mean KL	**0.00265**	0.00444	0.01096
Last 1,024, p99 KL	**0.04056**	0.06283	0.13641
Whole segment, mean KL	**0.10715**	0.13312	0.15758
Whole segment, p99 KL	**3.65623**	4.75603	5.55046
Answer region (27), mean KL	**0.00526**	0.00861	0.01155
Answer region, top-1	**96.30%**	92.59%	**96.30%**

Table 8: First cwe-128k text by region (Section 6.2's `--short-read` path). The KL rows are fp16-reference KL; *the top-1 row is target-token accuracy, not the fp16-argmax agreement of Table 7*, and is computed over 27 positions: $25/27 = 92.59\%$, $26/27 = 96.30\%$ — a difference of one position, not a result. The whole-segment p99 values are dominated by isolated hard positions (the measurement report lists single-position KLs of 17.3, 17.1 and 15.8); int8 reaches 3.66 as well, so they do not separate the arms.

separation, and not a stand-in for the three-arm gate. A full cell-to-row parity test remains open (A4).

8 Discussion: Where the Point Fits

8.1 A decision rule

The measured numbers compose into a short rule for operators. If the context sits below the roughly 28K-cell break-even, the ring costs more than the 4-bit base saves (Section 3.2); setup offers the window from 32K, where it is still cheaper than int8 (7.67 against 8.25 bit/element). At long context, quality-first workloads keep int8: it wins or ties every whole-segment statistic that carries weight on both 128K texts, and every region statistic at 2K and 4K (Tables 5–7). Memory-bound workloads at 128K take the window: 568 MiB freed, 416 more resident expert slots, prefill parity, decode inside spread (Table 4), and a measured KL penalty that is small on this task family and bounded by the 4K and 128K readings. Setup already follows this shape: int8 is the default, the window is opt-in from 32K upwards.

8.2 How firm is the grid

The neighborhood of the production grid is measured more thinly than the endpoints. On cwe-2k the production sink depth (R=64, S=256) is already statistically indistinguishable from int8 on both level-table bases [17]; the knee near S=512 sometimes quoted from the earlier rounds sits on the thinner R=8 arms of the 16-level base and must not be read as a statement about the shipped grid. At 4K the 512/128 pair scores the same quality as 256/64 (paired t=+0.02, Tables 5 and 6) for 7 MiB more and a slower decode, which is why 256/64 ships; at 128K the 512/128 grid was not quality-scored, so there is no quality statement for it there. What was not measured at 128K is anything smaller than 256/64: the shipping decision rests on the 4K quality tie plus the 128K runtime A/B, and on 256/64 being the cheapest 128K arm with a clean quality reading, not the local optimum. A finer sweep of S and R at 128K — and the knee move observed at 2K, where the load-bearing S sits higher — is the natural next measurement.

8.3 Cheaper overlays exist on paper

The window is orthogonal to the level table, and the workpackage trialed thinner level tables on the same mechanism early on [17]. Under the corrected byte model and the production sink depth the early reading shrinks: at the original R=8 sink depth on cwe-2k, the then-current byte model put a crossover near 4.8–5.3 bit/element where an 8-level-V windowed base looked byte-competitive; that crossover was withdrawn as an artifact of the too-small sink count (at R=64 the two bases were not separable across five shapes, $|t| \leq 1.5$), and the old byte model itself was superseded (the exact cell is additional; the span is R+S+B). What survives is that the window carries shorter level tables without measurable cost at R=64. A ≤3-bit base at 128K — the same mechanism at even fewer bytes — is unmeasured and is a concrete follow-up arm; its byte side must be priced with the corrected model.

8.4 Two unmeasured workload questions

Two questions bound how the point is used today. First, multi-turn growth: the window's home workload is a chat that grows by small appends to a long context, every turn after the first presumably reading large parts of the cache; the quality runs here score single segments, and the runtime A/B's mixed prompt set is the only evidence for repeated prompting. Second, mid-context degradation: the 128K corpus is scored at the tail, and the 4K corpus shows that per-cell quantization hurts where nothing protects the cell. A one-arm interior reading exists — the workpackage reports, for the window arm, KL 0.054 on the first 4,096 positions and 0.164 on one middle 4K block (positions 65,536–69,631) [17] — but the other arms are not scored on that block, so the paired mid-segment comparison is unmeasured. The first question is unmeasured outright. Neither argues for removing the window, but both belong on the list before larger deployment claims.

9 Threats to Validity

The measurements come from one GPU (RX 9060 XT, gfx1200), one operating system (Windows), one ROCm/HIP stack (7.1) and one weight set (Swift 1.5 IQ3_XXS); the tables name the engine build that produced them (Section 6.6). CUDA and SYCL were not built; the changes are backend-agnostic, but that is an argument, not a measurement. The short-context results have eight segments; each 128K result is one text with adjacent blocks, so those t values are descriptive. The $R{+}S{+}B$ ring is 20.3% of the 4K context at B=512 and 4.9% of the 128K context at B=6,144; those are the ring's resident capacity. A scored token under the short-read quality path sees the $R{+}S$ overlay (320 positions [17]), of which only the cells the indexer selects are read exact (Section 3, static reading); the 4K KL is an upper bound on the long-context benefit, not a prediction, and the 128K KL is measured separately. The prefill KV is also quantized, so the absolute KL gaps are not comparable with decode-only field numbers. Finally, the window is a memory trade-off, not a quality improvement over `int8`; the paper should be read that way. Two coverage limits stand out. The quality evidence comes from one task family (RULER complex word extraction) on one model; the full RULER set or LongBench subsets were not run, so the quality sentences here carry no claim outside this family. And no community-standard arm — for example a KIVI-style per-channel-K quantization with an fp16 residual [2, 3] — was measured on this engine, so the 5.29-bit point has no external anchor on the published trade-off curve; the fair version of that arm is per-channel on this indexer-architecture cache, not on a dense-attention cache. One intermittent engine hang was seen once on the *plain* 4-bit arm (no window) during a 128K prefill; it did not reproduce in 6/6 reruns, and the window arms completed three 128K runs without it — an open engine bug report, not evidence against the window.

10 Follow-up Work

Two gates remain. A full cell-to-row parity test should dump every window row at write time and re-read it through the window reader on every shape: ring wrap after scrolling past R rows, stale-tag fallback, sink rows, batch-boundary shapes and the empty-tag case. The second gate is build coverage: CUDA (nvcc) and Intel SYCL (dpcpp) should compile and pass their parity suites. `q4lm8v` is not offered on SYCL, so the window paths are dead there, but they should still compile.

Beyond the gates, the next measurement round has six limbs. Three need GPU time: a per-channel-K arm in the KIVI/KVQuant style on the same engine (the fair community competitor this paper does not include, Section 4); a second RULER task family at 4K, since the quality evidence above covers only cwe; and a finer S/R sweep at 128K including pairs smaller than 256/64, because the current data support 256/64 as the cheapest measured arm, not as a local optimum (Section 8.2). Two more score existing corpora differently: the paired mid-segment scoring of the middle 4K block (positions 65,536–69,631) on the 128K text, where the window arm is already scored but the other arms are not (Section 8.4), and a multi-turn append A/B where short prompts grow a long context turn by turn (Section 8.3). The last limb is offline arithmetic: a bootstrap over positions for the p99 entries, which needs no engine runs (Section 6.3). Optionally, the $\leq$3-bit overlay crossover at 128K (Section 8.4) rounds out the byte side. Both gates are review items, not shipped claims.

11 Conclusion

Hardening the exact-cell window was primarily an exercise in preventing silent failures at integration boundaries. Setup-side prevention and engine-side enforcement close the window-plus-streaming hole; a frozen grid closes state drift; sentinel tags close the empty-cell misread; a capped automatic chunk closes the mid-prompt abort. The default `int8` path stays byte-identical. The measurement-first validation shows the intended trade-off clearly: on the cwe corpora, the window removes most of the 4-bit base's KL error (73% of its gap to `int8` at 4K; 34–58% of the plain base's tail mean KL across the two 128K texts), approaches `int8` at 2K, and saves 36% of the KV memory at 128K. It does not beat `int8` in the tail regions that carry weight — the p99 and whole-segment numbers say so on both texts. That is the correct result for an opt-in memory point: keep `int8` as the quality default, offer the window where memory is the constraint, and measure both.

Two habits generalize beyond this feature. First, correctness is enforced where the failure would be silent: setup-side for users, engine-side for hand-edited configs, and a byte-identity gate for everything the default path touches. Second, an opt-in point earns its existence only against measured alternatives — here, against `int8` in regions that the mean KL alone cannot see. Both habits cost little and transfer to the next integration-boundary feature.

References

[1] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis. *Efficient Streaming Language Models with Attention Sinks*. arXiv:2309.17453, 2023 (ICLR 2024). Code: https://github.com/mit-han-lab/streaming-llm.

[2] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, X. Hu. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache*. ICML 2024, arXiv:2402.02750. Code: https://github.com/jy-yuan/KIVI.

[3] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami. *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization*. NeurIPS 2024, arXiv:2401.18079. Code: https://github.com/SqueezeAILab/KVQuant.

[4] H. Kang et al. *GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative LLM Inference*. arXiv:2403.05527, 2024.

[5] Y. Li et al. *SnapKV: LLM Knows What You Are Looking for Before Generation*. NeurIPS 2024, arXiv:2404.14469.

[6] Z. Cai et al. *PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling*. COLM 2025, arXiv:2406.02069.

[7] Z. Zhang et al. *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models*. NeurIPS 2023, arXiv:2306.14048.

[8] *QuaRot: Outlier-Free 4-Bit Inference in LLMs via Rotations*. arXiv:2404.00456, 2024.

[9] *RotateKV: Accurate and Robust 2-Bit KV Cache Quantization for Large Language Models*. IJCAI 2025 (proceedings 2025/0690).

[10] X. Li, Z. Xing, Y. Li, L. Qu, H. Zhen, Y. Yao, W. Liu, S. J. Pan, M. Yuan. *KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference*. ICML 2025, arXiv:2502.04420. Code: https://github.com/cmd2001/KVTuner.

[11] J. Zhang et al. *More Tokens, Lower Precision: Towards the Optimal Token-Precision Trade-off in KV Cache Compression*. arXiv:2412.12706, 2024.

[12] ggml-org, `llama.cpp` repository: KV-cache quantization types (`-cache-type-k/v`: fp16, q8_0, q4_0). https://github.com/ggml-org/llama.cpp.

[13] J. Kübler, E. Kurtić, et al. (vLLM team), *The State of FP8 KV-Cache and Attention Quantization in vLLM*, vLLM blog, 2026-04-22. https://vllm-project.github.io/2026/04/22/fp8-kvcache.html.

[14] Strata, `docs/DETAILS.md`: KV-cache quantization section (byte model, KL measurements, window 256/64 at 128K on both 128K texts). In-repo.

[15] Strata, `docs/QUANTS.md`: quant-quality yardsticks (mean KL, KL p99, top-1, perplexity) for the weight-quantization experiment; the cwe KL protocol is documented in the research-branch harness. In-repo.

[16] Strata, `bench/results/2026-09-27-kv-q4`: Q4_0 KV vs fp16/int8 document perplexity at 1K/8K (Q2_0). In-repo.

[17] Strata, `docs/KV_QUANT_WORKPACKAGE.md`: the window's runtime, quality and parity history (§7.12–§7.21). Branch `KV_QUANT_research` of the repository.

[18] Strata, `bench/results/2026-10-24-needle-window`: same-day needle pair, engine 0.1.47, `int8` against the 256/64 window, 6/6 and 6/6. In-repo.